Observability-Aware Trust-Based Grayhole Detection for Underwater Acoustic Networks

CS301 – Computer Networks Mini Project

Team 8

Department of Computer Science and Engineering
National Institute of Technology Karnataka, Surathkal

Team Members

Roll No.	Name	Email ID
241CS110	Appaji Nagaraja Dheeraj	appajidheeraj.241cs110@nitk.edu.in
241CS111	Aryan Bokolia	aryanbokolia.241cs111@nitk.edu.in
241CS117	Karthikeya Gupta	karthikeyagupta.241cs117@nitk.edu.in

Outline

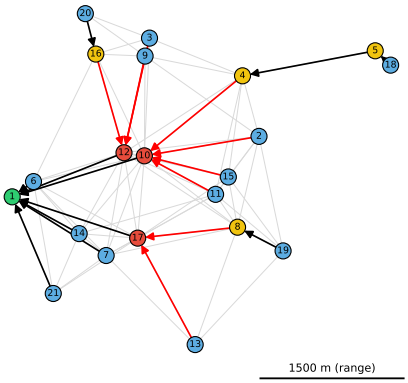
1. Introduction & Problem Statement
2. Background: Grayhole Attack and the Watchdog
3. Proposed Approach
4. Implementation in UnetStack
5. Results & Comparison
6. Conclusion & Future Work

Introduction & Problem Statement

- **Context:** Underwater Acoustic Sensor Networks (UASNs) carry sensor data to a surface sink over several acoustic hops (Network layer).
- **Problem:** A **grayhole** relay silently drops part of the packets it should forward; its loss blends with the normal loss of the channel.
- **Motivation:** The classic watchdog cannot tell a grayhole from an honest relay on a bad link, so it **accuses honest nodes** and routing around them costs delivery.
- **Objectives:**
  - Judge forwarding only when the monitor could actually observe it.
  - Compare observed forwards with what an honest relay *should* show on each link.
  - Evaluate against a fixed watchdog and a CAD-style detector in UnetStack.

Background: Grayhole Attack

- Every relay must forward its neighbours' packets to the **sink**.
- A **blackhole** drops everything and is easy to find.
- A **grayhole** drops only a fraction (here **30%**), so it looks like a lossy link.
- The **busiest relays** are made malicious: one grayhole can hurt many sensors.



Test network 201: sink (green), relays (yellow), grayholes (red)

Background: Why the Watchdog Fails Under Water

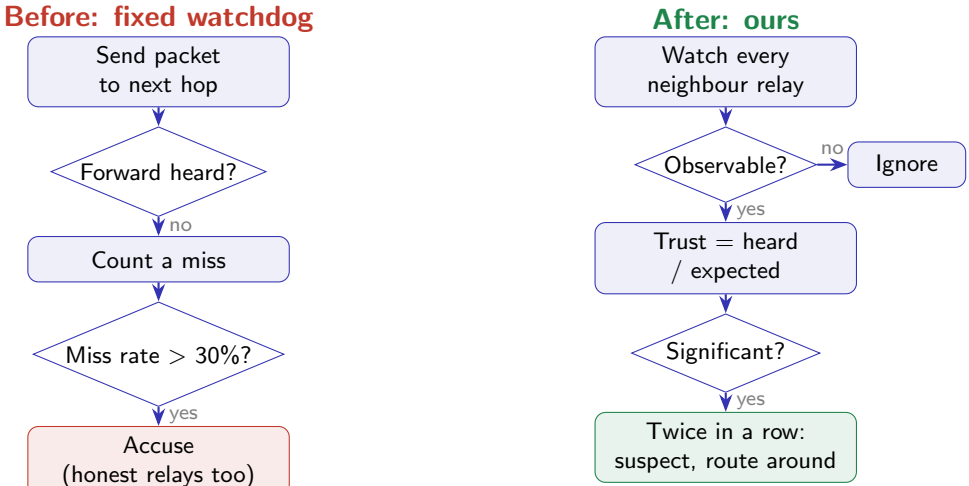
Watchdog (Marti *et al.* 2000; used in UnetStack3 by Chandavarkar & Suresh 2021): send a packet, listen for the next hop's forward, accuse if too many are missing.

A forward can be missing because . . .

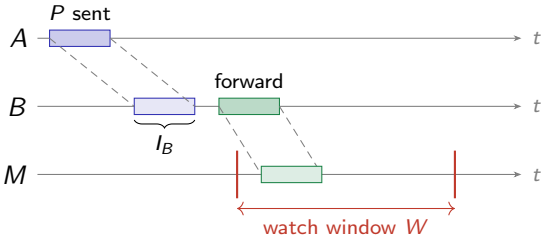
1	2	3	4
the relay **dropped** it (the attack)	the monitor was **transmitting** (half-duplex)	the packet **collided** at the relay	the **long link** lost it

Only cause 1 is an attack. Causes 2–4 **grow with traffic**, so a fixed threshold catches grayholes only by **also accusing honest relays**.

Proposed Approach: Baseline vs Ours



Proposed Approach: 1. Judge Only What Could Be Observed



From positions and the speed of sound (1500 m/s), monitor M knows when packet P reached relay B (I_B) and when B's forward should reach M (W).

A window counts only if

- **C1** M was silent during W
- **C3** B was not sending during I_B
- **C4** no known frame hit B during I_B

Otherwise the window is **ignored**, not counted as a miss.

Proposed Approach: 2. Trust Against the Expected Loss

- Channel loss grows with distance: $e(d) = 0.15\,(d/1500)^2$.
- Each kept window adds the chance an honest forward is heard: $g = (1 - e(d_{AB}))(1 - e(d_{BM}))$.
- Extra loss (collisions) is learned per **distance area** (near / medium / far) from the **other** relays' links: κ_{area}.

Trust of relay B

$$E = G\,(1 - \kappa_{\text{area}}) \qquad \text{expected forwards}$$

$$\text{Trust}_B = \frac{\sum \text{heard}}{\sum E}$$

$$\text{Estimated drop} = 1 - \text{Trust}_B$$

A long or busy link is *expected* to lose more, so it is not mistaken for a grayhole.

Proposed Approach: 3. Decide Carefully

Dynamic threshold

Flag only if ≥ 20 windows, drop $\geq 10\%$ and $z > z^*$; z^* adapts to the number of relays and this round's spread.

Two-round confirmation

A relay becomes a **suspect** only if flagged in **two 30-min rounds in a row**.

Baseline cleaning

Suspects and relays with $\geq 10\%$ drop leave the "normal loss", so several grayholes cannot hide each other.

Response

The sink gives the administrator a **ranked suspect list** and routes traffic **around** suspects wherever another path exists.

Implementation in UnetStack 7

- `MonitorNode`: one agent per node – traffic, grayhole drops, watch windows, reports.
- `Detectors`: fixed watchdog, CAD-style and ours.
- `Topology`: deployment and routes that avoid suspects.
- `DistanceLossChannel`: loss grows with distance.
- One pipeline `run.sh`: record → compare → live network → graphs.

```
onPacket(pkt) { // the attack
  if (dropProb > 0 && random() < dropProb)
    return // grayhole drops it
  transmit(pkt) // forward
}
onRound(t) { // at the sink, every 30 min
  rows = receivedEvidence() // delivered reports
  flagged = peel(rows, ALPHA, suspects).flagged
  suspects += flagged.intersect(lastFlagged)
  lastFlagged = flagged
  nextHop = routesAvoiding(suspects)
}
```

Experimental Setup

Parameter	**Value**
Simulator	UnetStack 7.0.0, protocol channel with distance loss, Java 8
Network	20 sensors + sink, 4 km $\times$ 4 km, range 1500 m (also 40-node, sparse)
Traffic	0.001–0.01 packets/s per sensor (0.005 default)
Grayholes	1, 2 or 3 busiest relays, dropping 30% (sweep 10–100%)
Fairness	thresholds tuned on topologies 101–106; tested on 201–210
Duration	6 h recorded runs, 12 h live-network runs, decision every 30 min
Compared	fixed watchdog (miss $>$ 30%), CAD-style ($z > 2$), ours

Same reports, confirmation and response for every method; only the detection rule differs.

Results at a Glance

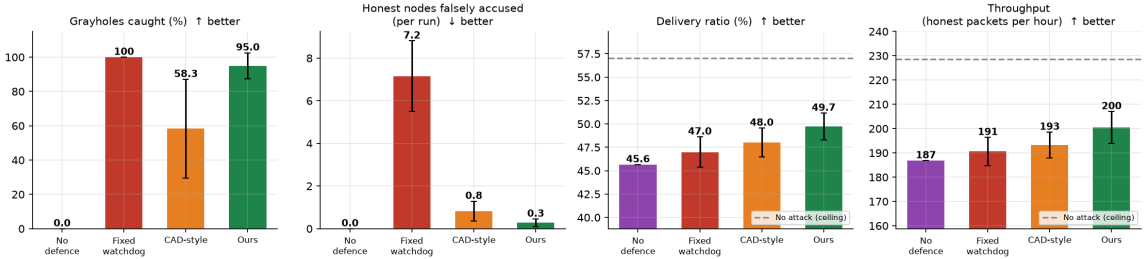
Live network, 12 h	Caught	Honest accused	Delivery	Throughput (pkt/h)
No attack	–	–	57.0%	228.6
Attack, no defence	0%	0	45.6%	186.8
Fixed watchdog	100%	7.2	47.0%	190.6
CAD-style	58%	0.8	48.0%	193.3
Ours	**95%**	**0.3**	**49.7%**	**200.5**

Key takeaway

Ours catches **95%** of grayholes while accusing **0.3** honest relays per run (fixed watchdog: **7.2**), and gives the highest delivery and throughput.

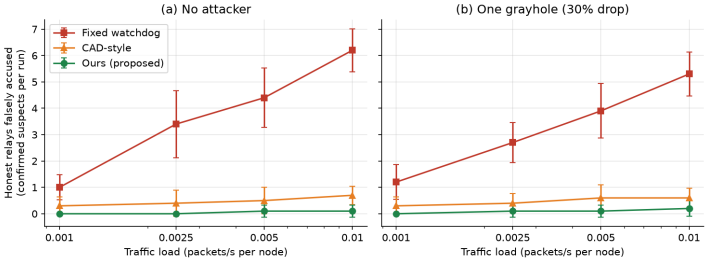
Results: Before vs After

Under attack: no defence → existing watchdogs → ours (same networks, same grayholes, same response)



The fixed watchdog catches everything but also accuses honest relays; routing around them wastes the gain.

Results: False Accusations Grow With Traffic

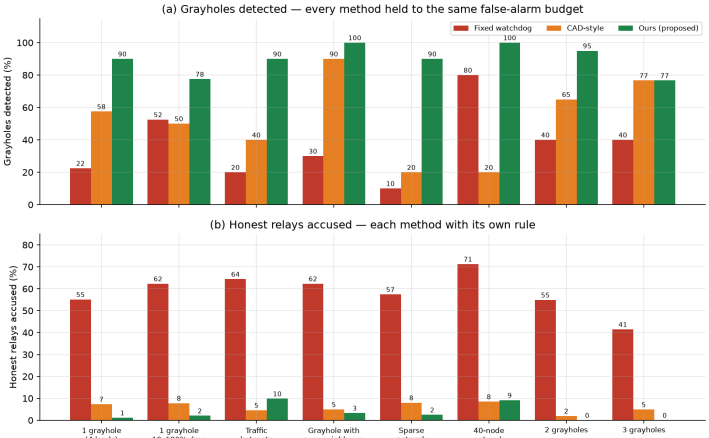


With **no attacker at all**:

- Fixed watchdog: 1.0 → 6.2 honest relays accused per run.
- CAD-style: 0.3–0.7.
- Ours: **0–0.1**.

More traffic = more collisions and deafness, which the watchdog counts as drops.

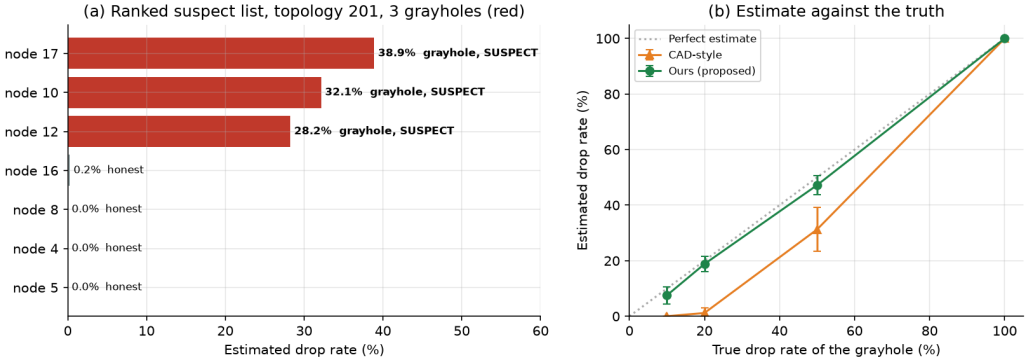
Results: Fair Contest at the Same False-Alarm Budget



Every threshold tuned so $\leq 5\%$ of honest relays are accused on the tuning networks.

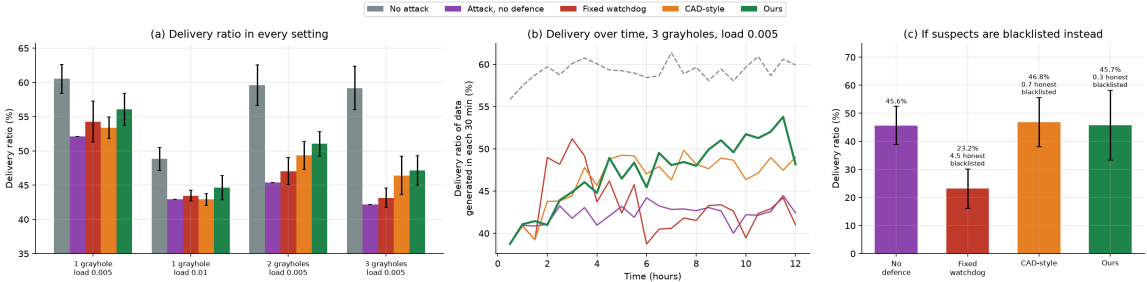
Grayholes	Fixed	CAD	**Ours**
1	22%	58%	**90%**
2	40%	65%	**95%**
3	40%	77%	**77%**

Results: What the Administrator Receives



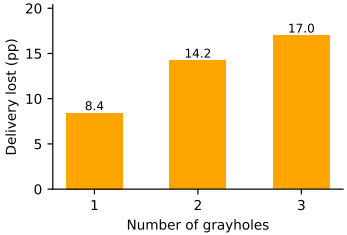
(a) The three grayholes rank first (28–39% estimated drop); honest relays sit at 0–0.2%.
(b) The estimate follows the true drop rate: 10/20/50% → 7.6/18.8/47.2%.

Results: Network Performance

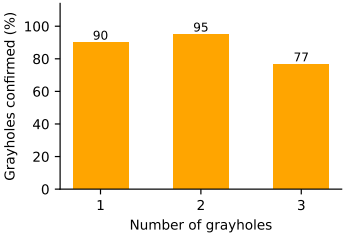


(a) Highest delivery in every setting: +1.7 to +5.6 points over no defence. (b) Delivery recovers once grayholes are confirmed. (c) If suspects are blacklisted, the watchdog's false accusations cut delivery to 23%.

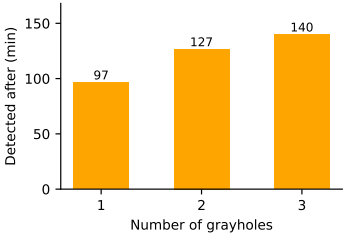
Results: Attack Strength and Detection Cost



Delivery lost to the attack



Grayholes confirmed by ours



Time until all are confirmed

More grayholes cost more delivery and take longer to confirm (two rounds = 60 min minimum).

Discussion

What each part contributes

- No baseline cleaning: 3-grayhole detection falls $23/30 \rightarrow 17/30$.
- No observability filter: similar detection, slightly more false accusations (1.5% vs 1.1%).

Where we are not clearly ahead

- Delivery gain over CAD-style is within the uncertainty.
- With 3 grayholes, CAD-style detects as often (77%).
- Hotspot and 40-node networks: $\approx$10% honest relays accused.

Conclusion & Future Work

Conclusion

- Under water, a missing forward is often **not** an attack.
- Observable windows + expected link loss + dynamic threshold + two-round confirmation: **95%** of grayholes caught, **0.3 vs 7.2** honest relays accused, highest delivery.
- Reproducible in UnetStack: `./run.sh demo` (3 min), `./run.sh` (all results).

Future Work

- Dishonest monitors, mobile nodes and location errors.
- Learn the loss curve online; larger and denser networks.

References & Thank You

1. B. R. Chandavarkar and N. Suresh, "Demonstration of selective forwarding attack and detection mechanism in UnetStack3," *ICCCNT*, 2021.
2. S. Marti, T. J. Giuli, K. Lai and M. Baker, "Mitigating routing misbehavior in mobile ad hoc networks," *ACM MobiCom*, 2000.
3. D. M. Shila, Y. Cheng and T. Anjali, "Mitigating selective forwarding attacks with a channel-aware approach in WMNs," *IEEE Trans. Wireless Commun.*, 2010.
4. I. F. Akyildiz, D. Pompili and T. Melodia, "Underwater acoustic sensor networks: research challenges," *Ad Hoc Networks*, 2005.
5. M. Chitre, R. Bhatnagar and W.-S. Soh, "UnetStack: an agent-based software stack and simulator for underwater networks," *IEEE OCEANS*, 2014.

Thank You!

Questions?